

A Parallel Geometric Multigrid Solver for the 3D Poisson Equation:

Algorithm, Implementation, and Extension Guide

Documentation for the **Multigrid** code base
(HPC for Mathematical Modelling, PhD course notes)

May 4, 2026

Abstract

This document describes the analytic problem, discretisation, and parallel multigrid algorithm implemented in the accompanying code base. The implementation solves the three-dimensional Poisson equation on the unit cube with homogeneous Dirichlet boundary conditions, using a node-based seven-point finite-difference Laplacian, a red-black Gauss–Seidel smoother with successive over-relaxation, and a geometric V-cycle multigrid acceleration on a halved-grid hierarchy. The code is parallelised with MPI through a Cartesian process topology and ghost-zone communication. We give a self-contained presentation aimed at a PhD-level audience in high-performance computing for mathematical modelling, then document the user-selectable parameters and explain how to extend the solver to other source terms and to non-homogeneous Dirichlet, Neumann, and Robin boundary conditions. Standard references are [2, 10, 7, 9, 8].

Contents

1	Introduction	2
2	The continuous problem	2
3	Discretisation	3
3.1	Node-based grid	3
3.2	Seven-point Laplacian	3
3.3	Index convention and storage	3
4	Stationary relaxation methods	4
4.1	Jacobi, Gauss–Seidel, SOR	4
4.2	Smoothing property of relaxation	4
4.3	Red-black ordering	4
5	Geometric multigrid	4
5.1	Two-grid idea	5
5.2	V-cycle and W-cycle	5
5.3	Transfer operators on a node-based grid	5
5.4	Convergence rate of the model V-cycle	6
6	Implementation	6
6.1	Source-tree layout	6
6.2	MPI domain decomposition	6

6.3	Smoother kernel: red-black Gauss–Seidel SOR	7
6.4	Defect	7
6.5	Hierarchy construction	7
6.6	Transfer operators	8
6.7	V-cycle	8
6.8	Driver loop	8
7	User-selectable parameters	9
7.1	Parameter reference	9
7.2	Practical tuning guidance	9
8	Extending the code	10
8.1	Different right-hand sides	10
8.2	Inhomogeneous Dirichlet, $u = g$ on $\partial\Omega$	11
8.3	Neumann, $\partial u/\partial n = q$ on $\partial\Omega$	11
8.4	Robin, $\alpha u + \beta \partial u/\partial n = r$	11
8.5	Periodic boundary conditions	12
8.6	Variable-coefficient or anisotropic operators	12

1 Introduction

The Poisson equation $-\Delta u = f$ is the canonical elliptic boundary-value problem and the standard test bed for elliptic solvers. Its symmetric, positive-definite, sparse algebraic structure — combined with the fact that classical relaxation methods (Jacobi, Gauss–Seidel, SOR) are very effective at damping high-frequency error components but very poor at damping low-frequency components — makes it the textbook target for *multigrid* methods, which combine a smoother on each level of a grid hierarchy with coarse-grid correction [1, 4, 7, 10]. Multigrid is the asymptotically optimal solver for the discrete Poisson problem: it reduces the algebraic error by a fixed factor in $\mathcal{O}(N)$ work, where N is the number of unknowns [2, Ch. 5].

This document describes a *parallel* geometric multigrid implementation written in C with MPI. It is organised as follows: Section 2 states the continuous problem; Section 3 the finite-difference discretisation; Section 4 reviews the relaxation methods that serve as smoothers; Section 5 develops the geometric multigrid theory needed to follow the implementation; Section 6 is a guided tour of the code; Section 7 documents the user parameters with practical tuning guidance; and Section 8 explains how to extend the code to other right-hand sides and boundary conditions.

2 The continuous problem

Let $\Omega = (0, 1)^3 \subset \mathbb{R}^3$. We seek $u: \bar{\Omega} \rightarrow \mathbb{R}$ satisfying

$$\Delta u(\mathbf{x}) = f(\mathbf{x}) \quad \text{in } \Omega, \quad u(\mathbf{x}) = 0 \quad \text{on } \partial\Omega, \quad (1)$$

where $\Delta = \partial_{xx} + \partial_{yy} + \partial_{zz}$ and $f \in L^2(\Omega)$. The associated weak formulation is: find $u \in H_0^1(\Omega)$ such that

$$\int_{\Omega} \nabla u \cdot \nabla v \, d\mathbf{x} = - \int_{\Omega} f v \, d\mathbf{x} \quad \forall v \in H_0^1(\Omega). \quad (2)$$

By the Lax–Milgram lemma this problem has a unique solution in $H_0^1(\Omega)$ [3, Ch. 6], and on a convex domain such as the unit cube the solution lies in $H^2(\Omega) \cap H_0^1(\Omega)$ with the elliptic-regularity estimate $\|u\|_{H^2} \leq C \|f\|_{L^2}$.

The driver currently uses the manufactured smooth solution

$$u_\star(\mathbf{x}) = \sin(\pi x) \sin(\pi y) \sin(\pi z), \quad f_\star(\mathbf{x}) = -3\pi^2 \sin(\pi x) \sin(\pi y) \sin(\pi z), \quad (3)$$

so that $\Delta u_\star = f_\star$ and $u_\star \equiv 0$ on $\partial\Omega$. This choice provides a closed-form reference for measuring the discretisation error.

Remark 1 (Sign convention). The implementation writes the equation as $\Delta u = f$ with $f = -3\pi^2 \sin(\pi x) \sin(\pi y) \sin(\pi z)$, not the more common form $-\Delta u = g$. The two are equivalent under $g = -f$. The *defect* computed by the code is $d = \Delta_h u_h - f_h$, where Δ_h denotes the discrete Laplacian; *not* $f_h - \Delta_h u_h$. This sign propagates through the multigrid update — in particular, the prolongation operator subtracts (rather than adds) the coarse correction (Section 6.6).

3 Discretisation

3.1 Node-based grid

We discretise $\bar{\Omega}$ with a uniform tensor-product grid

$$\mathbf{x}_{ijk} = (i h_x, j h_y, k h_z), \quad i = 0, \dots, N_x - 1, \quad j = 0, \dots, N_y - 1, \quad k = 0, \dots, N_z - 1,$$

where $N_x = n_x + 1$, $N_y = n_y + 1$, $N_z = n_z + 1$ and $h_x = 1/n_x$, $h_y = 1/n_y$, $h_z = 1/n_z$. The user specifies the cell counts n_x, n_y, n_z in the input file; the corresponding number of *grid points* per axis is one larger. The boundary nodes $i \in \{0, N_x - 1\}$ (and similarly j, k) are Dirichlet nodes and are not unknowns of the linear system.

3.2 Seven-point Laplacian

We approximate Δu by the standard second-order central difference

$$\Delta_h u_{ijk} := \frac{u_{i+1,j,k} - 2u_{ijk} + u_{i-1,j,k}}{h_x^2} + \frac{u_{i,j+1,k} - 2u_{ijk} + u_{i,j-1,k}}{h_y^2} + \frac{u_{i,j,k+1} - 2u_{ijk} + u_{i,j,k-1}}{h_z^2}. \quad (4)$$

Taylor expansion around $\mathbf{x}_{ijk}$ shows that, for $u \in C^4(\bar{\Omega})$, the truncation error satisfies

$$\Delta_h u(\mathbf{x}_{ijk}) - \Delta u(\mathbf{x}_{ijk}) = \frac{h_x^2}{12} \partial_x^4 u + \frac{h_y^2}{12} \partial_y^4 u + \frac{h_z^2}{12} \partial_z^4 u + \mathcal{O}(h^4), \quad (5)$$

so the scheme is second-order consistent in $h := \max(h_x, h_y, h_z)$ [8, Ch. 2]. Combined with discrete-maximum-principle stability, this yields second-order convergence in the ℓ^∞ norm, $\|u_h - u\|_{\ell^\infty(\Omega_h)} = \mathcal{O}(h^2)$ [8, Ch. 2–3].

The discrete system $A_h u_h = f_h$ has a symmetric, negative-definite, M -matrix structure with seven non-zeros per interior row and is the standard model problem in the multigrid literature [10, §2.1].

3.3 Index convention and storage

The code stores the unknowns in a flat array with the linear index

$$\text{idx}(i, j, k) = i + (j + k N_y) N_x,$$

so that i is the unit-stride (innermost) axis. This is the `gf_idx_3d` macro in `gf.h`. Each rank holds a contiguous tile of grid points plus a single layer of *ghost* cells on every face that is shared with another rank (Figure 1). The ghost width is fixed at $g_s = 1$, matching the seven-point stencil.

4 Stationary relaxation methods

Before turning to multigrid, we recall the building blocks the smoother is composed of. Let $A_h u_h = f_h$ be the discrete system from (4), with $A_h = D - L - U$ where D is the diagonal and L, U are the strict lower/upper-triangular parts.

4.1 Jacobi, Gauss–Seidel, SOR

The *Gauss–Seidel* (GS) iteration is $u^{(m+1)} = (D - L)^{-1}(Uu^{(m)} + f_h)$, i.e. each unknown is updated in turn using the most recent values of its neighbours. The *successive over-relaxation* (SOR) method introduces a relaxation parameter $\omega > 0$:

$$u_{ijk}^{(m+1)} = (1 - \omega) u_{ijk}^{(m)} + \omega \tilde{u}_{ijk}, \quad \tilde{u}_{ijk} = \frac{f_{ijk} - \sum_{(p,q,r) \neq (0,0,0)} a_{ijk;pqr} u_{i+p,j+q,k+r}}{a_{ijk;000}}, \quad (6)$$

where $\tilde{u}_{ijk}$ is the standard GS update. For SPD matrices, SOR converges if and only if $0 < \omega < 2$ [5, Thm. 10.1.2]; $\omega = 1$ recovers Gauss–Seidel.

For the constant-coefficient five-point Laplacian on an $n \times n$ square grid, Young’s classical analysis yields the optimal parameter

$$\omega_{\text{opt}} = \frac{2}{1 + \sin(\pi h)} \approx 2 - 2\pi h \quad \text{as } h \rightarrow 0, \quad (7)$$

with asymptotic spectral radius $\rho_{\text{opt}} = \omega_{\text{opt}} - 1 \approx 1 - 2\pi h$, i.e. a convergence rate of $\mathcal{O}(h)$ rather than the $\mathcal{O}(h^2)$ rate of plain Gauss–Seidel [12, Ch. 5], [9, §4.2.2], [11, Ch. 3]. In three dimensions the analogous formula has the same leading behaviour [10, §2.5].

4.2 Smoothing property of relaxation

The reason classical relaxation appears in multigrid is not its global convergence rate but its *smoothing* property: the high-frequency components of the error are damped quickly even when the low-frequency components are not. For the discrete Laplacian, expand the error $e^{(m)} = u_h - u^{(m)}$ in the Fourier basis. The Jacobi/GS/SOR amplification factor depends on the wavenumber θ . For weighted Jacobi and Gauss–Seidel applied to the model problem, the smoothing factor (the worst amplification factor over high-frequency modes $\theta \in [\pi/2, \pi]$) is bounded away from 1 uniformly in h [10, §2.1], [2, §3.2]. This is the property that makes multigrid work.

4.3 Red-black ordering

The Gauss–Seidel iteration is sequential as written, because updating u_{ijk} requires the already-updated $u_{i-1,j,k}$, $u_{i,j-1,k}$, $u_{i,j,k-1}$. For the seven-point stencil this dependency disappears under a *red-black* (or *checkerboard*) ordering: we colour each grid point with parity $c(i, j, k) = (i + j + k) \bmod 2$, then update all red points first (every red neighbour stencil consists only of black points, which are not being updated), exchange ghosts, and finally update all black points. The resulting iteration is a genuine Gauss–Seidel iteration with respect to the red-black permutation, but each half-sweep is fully parallel. For the five-point Laplacian, red-black GS has smoothing factor exactly 1/4 in two dimensions, independent of h [10, §3.2]; the three-dimensional seven-point case has the same qualitative behaviour.

5 Geometric multigrid

This section condenses the parts of the multigrid story that are needed to read the code; the canonical references are [2, 10, 7].

5.1 Two-grid idea

Suppose we have a fine grid Ω_h with operator A_h and a coarse grid Ω_H (with $H = 2h$) with operator A_H , plus

- a *restriction* $R_h^H : \Omega_h \rightarrow \Omega_H$, transferring fine-grid functions to the coarse grid;
- a *prolongation* $P_H^h : \Omega_H \rightarrow \Omega_h$, transferring coarse-grid functions to the fine grid.

After a few sweeps of relaxation on $A_h u_h = f_h$, the error $e = u_h^* - u_h$ is smooth and therefore well representable on Ω_H . The two-grid cycle reads:

1. Pre-smooth: $u_h \leftarrow S_h^{\nu_1}(u_h, f_h)$.
2. Compute the defect $d_h = A_h u_h - f_h$ (note the sign convention of Section 2).
3. Restrict: $d_H = R_h^H d_h$.
4. Solve the coarse defect equation $A_H e_H = d_H$ (exactly or approximately).
5. Prolong and *subtract* the correction: $u_h \leftarrow u_h - P_H^h e_H$.
6. Post-smooth: $u_h \leftarrow S_h^{\nu_2}(u_h, f_h)$.

5.2 V-cycle and W-cycle

Recursively replacing the exact solve in step 4 with γ applications of the same cycle defines the *multigrid cycle of index γ* : $\gamma = 1$ is a V-cycle, $\gamma = 2$ is a W-cycle [10, Fig. 3.1]. In the present code the parameter `subcycles` controls γ at every level (see Section 6.7).

Theorem 2 (Multigrid optimality, informal). *For the discrete Poisson problem with full-weighting restriction, linear interpolation, and a sufficient number of relaxation sweeps, the V-cycle reduces the error by a factor that is bounded away from 1 uniformly in h . The cost of one V-cycle is $\mathcal{O}(N)$ where N is the number of fine-grid unknowns, so the total cost of obtaining a discretisation-accurate solution is $\mathcal{O}(N)$.*

A precise statement and proof under the appropriate regularity and approximation hypotheses is given in [7, Ch. 6] and [10, Ch. 7].

5.3 Transfer operators on a node-based grid

Restriction. We use *full-weighting* restriction with the $3 \times 3 \times 3$ stencil

$$(R_h^H d)_{IJK} = \frac{1}{64} \sum_{p,q,r \in \{-1,0,1\}} w_{pqr} d_{2I+p, 2J+q, 2K+r}, \quad (8)$$

with weights $w_{000} = 8$, $w = 4$ on the six face-adjacent points, $w = 2$ on the twelve edge-adjacent points, and $w = 1$ on the eight corners (total weight 64). This is the natural generalisation to three dimensions of the canonical 2D full-weighting operator and is the standard choice for the Poisson problem [2, §3.5], [10, §2.3].

Prolongation. We use *trilinear* interpolation: a fine-grid point at parity $(p_x, p_y, p_z) \in \{0, 1\}^3$ relative to its coarse cell is set to the average of the $2^{p_x+p_y+p_z}$ surrounding coarse-grid points. Concretely, with $p_x + p_y + p_z = 0, 1, 2, 3$ this gives copy, two-point average, four-point average, eight-point average respectively. Trilinear prolongation is the formal adjoint of full-weighting restriction up to a factor of 8, so the pair (R_h^H, P_H^h) satisfies the variational property $P_H^h = c(R_h^H)^T$ that underlies most multigrid convergence proofs [10, §2.3.4].

Injection. At coincident points (i.e. at every coarse-grid node, which by construction lies on the fine grid as well), the *injection* operator simply copies the fine value. In this code injection is used to seed the coarse-grid right-hand side at the boundary nodes, after which restriction overwrites the interior nodes; this avoids special-casing of the restriction stencil at boundary points.

5.4 Convergence rate of the model V-cycle

For the constant-coefficient d -dimensional Laplacian discretised by the standard $(2d+1)$ -point stencil, with red-black Gauss–Seidel smoothing, full-weighting restriction, and linear (trilinear in 3D) interpolation, the asymptotic V-cycle convergence factor is bounded above by approximately 0.1–0.2 depending on the number of pre/post-smoothing sweeps; the factor is independent of h [10, §4]. Local Fourier analysis (LFA) [10, Appx. A] is the standard tool for quantifying these rates.

6 Implementation

6.1 Source-tree layout

The code is layered. From bottom to top:

`domain.{c,h}`

MPI Cartesian decomposition. The function `automatic_topology` chooses the process grid (p_x, p_y, p_z) by greedy prime factorisation of the rank count; `setup_3d_domain` builds the Cartesian communicator and computes per-rank local extents and global offsets.

`gf.{c,h}`

Grid-function containers. `struct ngfs_3d` holds the local sizes, origin, spacing, an array of variable slots (`vars[]`), pre-allocated face-buffers for ghost exchange, and parent/child pointers used to chain multigrid levels.

`comm.{c,h}`

Ghost-zone synchronisation. `sync_var_3d` packs faces, posts non-blocking sends/receives one axis at a time, and unpacks.

`gauss_seidel.{c,h}`

The smoother and the defect operator: `gauss_seidel_3d`, `calc_defect_3d`, `apply_bc_3d`.

`multigrid.{c,h}`

Hierarchy construction (`ngfs_3d_create_hierarchy`), transfer operators (`inject_var_3d`, `restrict_var_3d`, `prolong_var_3d`), and the V-cycle driver `vcycle_3d`.

`io.{c,h}`

Per-rank JSON output.

`parameter.{cc,h}` and `toml.hpp`

TOML parameter parsing. `parameter.cc` is C++; the `parse_parameter_file` entry point is exposed as `extern "C"` so the C driver can link against it.

`driver_multigrid.c`

The top-level program: parameter parsing, grid set-up, RHS initialisation from (3), the outer iteration loop, error reporting, and JSON output.

A small set of variable indices is fixed in `gauss_seidel.h`: `VAR_SOL = 0` (the solution u), `VAR_RHS = 1` (the source f), `VAR_DEF = 2` (the defect $d = \Delta_h u - f$).

6.2 MPI domain decomposition

Given P MPI ranks and global grid dimensions (N_x, N_y, N_z) , the function `automatic_topology` computes a process grid (p_x, p_y, p_z) with $p_x p_y p_z = P$ via a greedy descent on the prime factors of P , biasing each factor toward the longest current axis to keep tile aspect ratios close to one.

`MPI_Cart_create` produces the Cartesian communicator used everywhere downstream. The periodicity flags are zero in all directions, which matches the Dirichlet boundary conditions; switching to periodic boundaries requires changing this in `setup_3d_domain` (see Section 8.5).

A face is a *physical boundary* of the global domain on this rank when the corresponding neighbour rank does not exist (`lower_x_rank == INVALID_RANK` and analogous). Boundary conditions are applied only on those faces; faces shared with another rank are handled by ghost exchange.

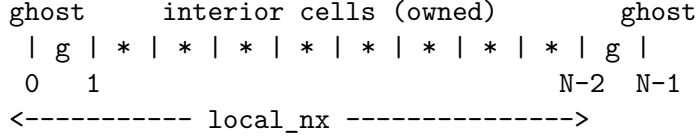


Figure 1: One-dimensional view of a per-rank tile. The width- $g_s = 1$ ghost layer on each side stores values owned by the neighbouring rank (or the physical boundary). The seven-point stencil at any interior point reads at most one ghost cell on each axis.

6.3 Smoother kernel: red-black Gauss–Seidel SOR

The function `gauss_seidel_3d` (`gauss_seidel.c:86`) implements (6) in red-black ordering with global parity $c(i, j, k) = (g_i + g_j + g_k) \bmod 2$, where $g_i = \text{local_i0} + i$ is the global index of local point i . Using the global parity — not the local parity — guarantees that neighbouring ranks colour shared-face neighbours consistently, regardless of where rank boundaries fall. Each iteration consists of a red sweep, a ghost exchange and BC application, a black sweep, and another ghost+BC sync.

The stencil coefficients in the GS update are precomputed once per level from h_x, h_y, h_z :

$$D = 2(h_x^2 h_y^2 + h_x^2 h_z^2 + h_y^2 h_z^2),$$

$$c_x = h_y^2 h_z^2 / D, \quad c_y = h_x^2 h_z^2 / D, \quad c_z = h_x^2 h_y^2 / D, \quad c_s = h_x^2 h_y^2 h_z^2 / D,$$

so that $\tilde{u}_{ijk} = c_x(u_{i\pm 1, j, k}) + c_y(u_{i, j\pm 1, k}) + c_z(u_{i, j, k\pm 1}) - c_s f_{ijk}$. The SOR update (6) is then $u_{ijk} \leftarrow (1 - \omega)u_{ijk} + \omega \tilde{u}_{ijk}$.

6.4 Defect

`calc_defect_3d` (`gauss_seidel.c:196`) evaluates $d_{ijk} = (\Delta_h u)_{ijk} - f_{ijk}$ at every interior point, zeros it on physical boundaries (so the defect is consistent with Dirichlet BCs), reduces the local maximum modulus across the Cartesian communicator with `MPI_Allreduce(..., MPI_MAX)`, and synchronises the ghost layer of `VAR_DEF` so that the restriction stencil can read complete data on the next call.

6.5 Hierarchy construction

`ngfs_3d_create_hierarchy` (`multigrid.c:228`) builds the parent→child linked list by repeatedly halving the cell counts in each direction and doubling the spacings. The same Cartesian process grid is used at every level; coarsening reduces the per-rank tile but never agglomerates ranks. Coarsening stops when either:

- (i) the cell count in any direction is odd (so exact 2:1 coarsening is impossible), or
- (ii) the minimum number of interior cells per rank per direction, reduced across all ranks via `MPI_Allreduce`, drops below the user-supplied `min_cells` threshold.

For maximum hierarchy depth the user should choose each n_x, n_y, n_z of the form $2^p \cdot c$ with c odd; the algorithm then reaches a coarsest grid of $c + 1$ points per axis (or stops earlier on the `min_cells` criterion).

6.6 Transfer operators

The injection, restriction, and trilinear prolongation operators of Section 5.3 are implemented in `inject_var_3d`, `restrict_var_3d`, `prolong_var_3d`. Two implementation details are worth highlighting:

- *Boundary handling in restriction.* The full-weighting stencil is applied only at interior coarse-grid points. Coarse boundary points are first overwritten by injection from the fine-grid boundary, ensuring that homogeneous Dirichlet data on the fine grid carries through unchanged.
- *Sign of the prolongation.* Because the code's defect convention is $d = \Delta_h u - f$ rather than $f - \Delta_h u$, the coarse correction e_H is the solution of $A_H e_H = d_H$ (not $A_H e_H = -d_H$), and the fine-grid update is $u_h \leftarrow u_h - P_H^h e_H$. This is why `prolong_var_3d` performs `pval[idx] -= update`. Physical boundary fine-grid points are skipped to preserve the Dirichlet data set by `apply_bc_3d`.
- *Ghost requirement.* The trilinear stencil reads up to one neighbouring coarse-grid point in each direction, so the coarse correction's ghost layer must be valid before `prolong_var_3d` is called. In practice this is guaranteed because the smoother (which is the last operation on the coarse level before the recursion returns) leaves the ghost layer of `VAR_SOL` synchronised.

6.7 V-cycle

`vcycle_3d` (`multigrid.c:711`) implements Listing 1.

```

1 vcycle_3d(level, n_smooth, omega, tol, subcycles):
2     gauss_seidel_3d(level, n_smooth, omega)           // pre-smoothing
3     norm = calc_defect_3d(level)                       // VAR_DEF, ghosts synced
4     if level.child != NULL and norm > tol:
5         for it = 1 .. subcycles:
6             zero level.child.VAR_SOL; apply_bc_3d
7             inject_var_3d(level, VAR_DEF, child, VAR_RHS)
8             restrict_var_3d(level, VAR_DEF, child, VAR_RHS)
9             vcycle_3d(child, n_smooth, omega, tol, subcycles)
10            sync_var_3d(level, VAR_SOL)                 // ghosts of fine SOL
11            gauss_seidel_3d(level, n_smooth, omega)     // post-smoothing
12            gauss_seidel_3d(level, n_smooth, 1.0)       // extra plain GS pass
13            norm = calc_defect_3d(level)
14            if norm <= tol: break
15 if level.parent != NULL:
16     prolong_var_3d(level, VAR_SOL, level.parent, VAR_SOL) // u -= e
17 return norm

```

Listing 1: V-cycle pseudocode (matches `vcycle_3d`).

With `subcycles = 1` this is the standard V-cycle. With `subcycles > 1` the inner loop performs additional coarse-grid visits per level until the defect tolerance is met or the budget is exhausted, producing a behaviour intermediate between a V-cycle and a W-cycle.

6.8 Driver loop

The driver in `driver_multigrid.c` reads the TOML file, allocates the root grid with three variable slots, optionally builds the hierarchy, initialises `VAR_RHS` from (3) and `VAR_SOL = 0`, then iterates up to `n_iters` times. Each iteration is either one V-cycle (if `multigrid = true`) or one call to the smoother followed by a defect computation; the loop terminates early if $\|d\|_{L^\infty} < \text{tol}$. After convergence the driver computes $\|u_h - u_\star\|_{L^\infty}$ on the local interior, reduces across ranks with `MPI_MAX`, and prints the result. Per-rank JSON output is written by `output_3d_gf`.

7 User-selectable parameters

The driver takes a single command-line argument: the path to a TOML parameter file. An example (`multigrid.toml`) is shipped with the code:

```

1 [grid]
2 nx_cells = 256
3 ny_cells = 256
4 nz_cells = 256
5
6 [solver]
7 multigrid = true
8 omega     = 1.5
9 n_smooth  = 100
10 n_iters   = 20
11 tol       = 1.0e-9
12 subcycles = 1
13 min_cells = 4

```

Listing 2: Sample input file.

7.1 Parameter reference

Section / key	Type	Meaning
<code>[grid].nx_cells</code>	positive int	Cells in x ; grid points = n_x+1 . For deepest coarsening, prefer $n_x = 2^p \cdot c$ with c odd.
<code>[grid].ny_cells</code>	positive int	Cells in y .
<code>[grid].nz_cells</code>	positive int	Cells in z .
<code>[solver].multigrid</code>	bool	true : V-cycle multigrid. false : single-grid red-black Gauss–Seidel SOR.
<code>[solver].omega</code>	float $\in (0, 2)$	SOR relaxation parameter. $\omega = 1$ is plain Gauss–Seidel; $1 < \omega < 2$ is over-relaxation. See Section 7.2.
<code>[solver].n_smooth</code>	positive int	Number of red-black GS iterations per smoothing call.
<code>[solver].n_iters</code>	positive int	Maximum number of outer iterations (V-cycles, or single-grid GS calls).
<code>[solver].tol</code>	non-negative float	Termination threshold on $\ d\ _{L^\infty}$.
<code>[solver].subcycles</code>	positive int	(multigrid only) Maximum coarse-grid visits per level. $1 = \text{V-cycle}$, > 1 behaves like a partial W-cycle.
<code>[solver].min_cells</code>	positive int	(multigrid only) Minimum interior cells per rank per direction. Coarsening stops below this.

Table 1: Input parameters.

7.2 Practical tuning guidance

ω (SOR parameter). For multigrid, the smoother only needs to damp high frequencies; a common and robust choice is $\omega \in [1.0, 1.3]$ for red-black GS. Higher values (closer to the single-grid optimum (7)) sometimes accelerate the smoother but can erode the smoothing factor. When running in single-grid mode, choose ω near ω_{opt} ; for $h \approx 1/256$ this gives $\omega \approx 1.97$. The shipped default $\omega = 1.5$ is a compromise that works for both modes on the example problem.

`n_smooth`. Multigrid theory predicts that a small number of smoothing sweeps ($\nu_1 = \nu_2 = 1$ or 2) is sufficient [10, §4]. In practice, on a parallel machine where ghost exchanges have non-trivial latency, larger `n_smooth` (between 4 and the shipped 100) can improve *strong-scaling* efficiency

by amortising the communication cost even though it does not change the asymptotic algorithmic complexity. Tune for time-to-solution, not iteration count.

n_iters and tol. Set `tol` to the target precision (typically a small multiple of the discretisation error $\mathcal{O}(h^2)$) and `n_iters` to a generous upper bound; the loop exits early once the tolerance is met.

subcycles. Leave this at 1 unless you observe that V-cycles plateau before the defect drops to `tol`; in that case increasing it to 2 behaves like a W-cycle on the affected levels and is the cheapest remedy.

min_cells. A small value (the default 4) lets the hierarchy go as deep as the grid allows. On a very large rank count, 4 may be too aggressive: the coarsest level becomes communication-bound because each rank holds only a handful of cells. In that regime, 8 or 16 is often a better choice. At present the code does not implement rank-agglomeration on coarse grids, which is the more sophisticated remedy [10, §6.3].

nx_cells, ny_cells, nz_cells. Powers of two maximise hierarchy depth. Cell counts that are not powers of two are accepted but bottom out at the first odd factor.

8 Extending the code

The grid hierarchy, the smoother coefficients (which are computed per level from h_x, h_y, h_z), the defect, and the transfer operators are *independent of* the right-hand side and depend on the boundary conditions only through the function `apply_bc_3d` and the boundary handling inside `prolong_var_3d`. Modifications are therefore highly localised.

8.1 Different right-hand sides

To solve $\Delta u = f$ for a different f on the same domain with the same homogeneous Dirichlet condition, edit only the RHS-init loop in `driver_multigrid.c` (lines 125–139):

```

1 for (int64_t k = 0; k < gfs.nz; k++) {
2     const double z = gfs.z0 + k * gfs.dz;
3     for (int64_t j = 0; j < gfs.ny; j++) {
4         const double y = gfs.y0 + j * gfs.dy;
5         for (int64_t i = 0; i < gfs.nx; i++) {
6             const double x = gfs.x0 + i * gfs.dx;
7             const int64_t idx = gf_idx_3d(&gfs, i, j, k);
8             rhs[idx] = my_f(x, y, z);    // <-- replace with new source
9             sol[idx] = 0.0;
10        }
11    }
12 }
```

The closed-form-error block (lines 168–185) should be removed unless a manufactured solution is available. Nothing else needs to change.

A cleaner refactor is to introduce a function pointer `double (*f_rhs)(double,double,double)` (and optionally `double (*u_exact)(...)`) that the driver dispatches on. This is a recommended student exercise.

8.2 Inhomogeneous Dirichlet, $u = g$ on $\partial\Omega$

The function `apply_bc_3d` (`gauss_seidel.c:21`) currently writes 0 on every physical-boundary face. To enforce $u = g$, replace those writes with the value of g at the corresponding global coordinate, e.g.

```

1 if (gfs->domain.lower_x_rank == INVALID_RANK)
2     for (int k = 0; k < nz; k++)
3         for (int j = 0; j < ny; j++) {
4             const double y = gfs->y0 + j * gfs->dy;
5             const double z = gfs->z0 + k * gfs->dz;
6             v[gf_indx_3d(gfs, 0, j, k)] = g_lower_x(y, z);
7         }

```

Crucially: only the *fine grid* requires inhomogeneous Dirichlet data. On coarser levels the unknown is the *correction* e_H , whose boundary condition is homogeneous ($e = 0$ on $\partial\Omega$) regardless of the original problem. The code already enforces this implicitly: `vcycle_3d` zeroes the child's `VAR_SOL` before recursion, and `prolong_var_3d` skips physical boundary points so the fine boundary values are preserved across coarse-grid corrections. The recommended pattern is therefore

- add a function `apply_bc_3d_inhomog` that writes g on physical boundaries and call it once for the *root-level* `VAR_SOL` during initialisation;
- leave the existing `apply_bc_3d` (with homogeneous values) in place where it is invoked by the smoother and the V-cycle.

8.3 Neumann, $\partial u / \partial n = q$ on $\partial\Omega$

Neumann boundary conditions are imposed at the discrete level by introducing *ghost rows* just outside the physical domain and setting them so that the centred difference of the normal derivative matches q . For the lower- x boundary this gives

$$\frac{u_{1,j,k} - u_{-1,j,k}}{2h_x} = q(\mathbf{x}_{0,j,k}) \quad \implies \quad u_{-1,j,k} = u_{1,j,k} - 2h_x q(\mathbf{x}_{0,j,k}). \quad (9)$$

In the present code the “ghost row outside the physical boundary” is exactly the cell at local index $i = 0$ when the rank is on the lower- x face (because that face has no MPI neighbour to fill the layer). Implementing Neumann therefore amounts to:

1. In `apply_bc_3d`, instead of zeroing the boundary face, reflect the interior with the offset above, e.g. `v[i=0] = v[i=2] - 2*hx*q(...)` at lower- x , and the symmetric form at upper- x .
2. Treat the boundary points as *unknowns*: extend the sweep range in `gauss_seidel_3d` and `calc_defect_3d` to include $i = 0$ and $i = N_x - 1$ on physical-boundary ranks. This requires modifying the loop bounds; the simplest approach is to keep the interior-only sweeps but follow them by an explicit ghost-row update via (9) that mimics the boundary equation.
3. Remove the corresponding `continue` guards in `prolong_var_3d`: with Neumann BCs the boundary correction is non-zero.

Pure Neumann conditions render the operator singular up to a constant (the constant function lies in the null space). In that case the solution is determined only up to an additive constant, and the discrete RHS must satisfy the discrete compatibility condition $\int f \, d\mathbf{x} = \int q \, dS$, i.e. the sum of the discrete RHS plus the boundary flux contributions must vanish; otherwise the system has no solution. In practice one (i) projects the RHS onto the mean-zero subspace each iteration, and (ii) anchors the solution by fixing u at one node, or by projecting u onto the mean-zero subspace each iteration as well. See [2, §6.4] and [10, §5.6] for a thorough discussion.

8.4 Robin, $\alpha u + \beta \partial u / \partial n = r$

Robin (or third-kind) boundary conditions interpolate between Dirichlet ($\beta = 0$) and Neumann ($\alpha = 0$) and are implemented by combining the two ghost-row strategies above. At the lower- x

face,

$$\alpha u_{0,j,k} + \beta \frac{u_{1,j,k} - u_{-1,j,k}}{2h_x} = r(\mathbf{x}_{0,j,k}),$$

which solves for the ghost value

$$u_{-1,j,k} = u_{1,j,k} + \frac{2h_x}{\beta} [\alpha u_{0,j,k} - r(\mathbf{x}_{0,j,k})]. \quad (10)$$

Because $u_{-1,j,k}$ depends on $u_{0,j,k}$, the boundary equation is typically eliminated by substituting (10) into the seven-point stencil at $i = 0$ and absorbing the $\alpha u_{0,j,k}$ term into the diagonal. Concretely, the modified equation for the lower- x boundary point becomes

$$\frac{2u_{1,j,k} - 2u_{0,j,k} - \frac{2h_x\alpha}{\beta}u_{0,j,k} + \frac{2h_x}{\beta}r}{h_x^2} + (y \text{ and } z \text{ second differences}) = f_{0,j,k}, \quad (11)$$

i.e. a modified diagonal coefficient $-2/h_x^2 - 2\alpha/(h_x\beta)$ and an additional contribution to the RHS of $-2r/(h_x\beta)$.

In the code this is implemented by:

1. Replacing `apply_bc_3d` on Robin faces with the ghost-update (10).
2. Adding boundary-stencil branches to `gauss_seidel_3d` and `calc_defect_3d` that use (11) on Robin-boundary points. A clean way to organise this is to precompute, on each rank that owns a Robin face, modified coefficient arrays (or a flat list of boundary-equation “corrections”) so that the inner kernels remain branch-free.
3. Modifying `prolong_var_3d` to interpolate the correction at boundary points (Robin corrections are non-zero on the boundary).

Note that on coarser levels the BC stays Robin with the *same* α and β but the homogeneous data $r = 0$ for the coarse-grid correction, exactly as in the inhomogeneous-Dirichlet case (Section 8.2). Robin boundary conditions are also discussed in [10, §2.3.2] in the multigrid context.

8.5 Periodic boundary conditions

Periodicity is essentially free: it amounts to (i) flipping the `periods` flags passed to `MPI_Cart_create` in `setup_3d_domain` from 0 to 1 on the periodic axes (the ghost exchange machinery in `sync_var_3d` then automatically wraps), (ii) skipping the corresponding writes in `apply_bc_3d`, and (iii) removing the boundary `continue` guards in `prolong_var_3d` on the periodic axes. Pure-periodic problems share the singular-operator issue with pure-Neumann and require the same compatibility-and-projection treatment.

8.6 Variable-coefficient or anisotropic operators

For $\nabla \cdot (a(\mathbf{x})\nabla u) = f$ or strongly anisotropic operators ($h_x \ll h_y$), the geometric multigrid components in this code may degrade. Practical extensions include:

- rewriting the inner kernels of `gauss_seidel_3d` and `calc_defect_3d` to use the new stencil (the colouring, ghost exchange, and BC handling carry over unchanged);
- using a *Galerkin* coarse operator $A_H = R_h^H A_h P_H^h$ instead of re-discretising on each level [10, §5.4], which requires storing the operator on every level rather than just the spacings;
- considering algebraic multigrid (AMG) for problems where the geometric coarsening hierarchy is poorly matched to the operator [10, Appx. A].

Acknowledgements and further reading

The presentation here closely follows [2, 10]; the parallel considerations are standard MPI material as in [6]; the finite-difference background is [8]. For deeper analytical foundations of multigrid the reader is referred to [7].

References

- [1] A. Brandt. Multi-Level Adaptive Solutions to Boundary-Value Problems. *Mathematics of Computation*, 31(138):333–390, 1977.
- [2] W. L. Briggs, V. E. Henson, and S. F. McCormick. *A Multigrid Tutorial*. SIAM, Philadelphia, 2nd edition, 2000. ISBN 978-0-89871-462-3.
- [3] L. C. Evans. *Partial Differential Equations*. American Mathematical Society, 2nd edition, 2010.
- [4] R. P. Fedorenko. The speed of convergence of one iterative process. *USSR Computational Mathematics and Mathematical Physics*, 4(3):227–235, 1964.
- [5] G. H. Golub and C. F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.
- [6] W. Gropp, E. Lusk, and A. Skjellum. *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. MIT Press, 3rd edition, 2014.
- [7] W. Hackbusch. *Multi-Grid Methods and Applications*. Springer-Verlag, Berlin, 1985.
- [8] R. J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM, Philadelphia, 2007.
- [9] Y. Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, Philadelphia, 2nd edition, 2003.
- [10] U. Trottenberg, C. W. Oosterlee, and A. Schüller. *Multigrid*. Academic Press, San Diego, 2001.
- [11] R. S. Varga. *Matrix Iterative Analysis*. Springer, second revised and expanded edition, 2000.
- [12] D. M. Young. *Iterative Solution of Large Linear Systems*. Academic Press, New York, 1971.